

HERMES RE-AUDIT GUVENLIK RAPORU

Tarama Tarihi:	2026-07-06 14:19:24
Kontrat Adresi:	0x4200
Ag Katmani:	Base Blockchain (ChainID: 8453)

GENEL GUVENLIK SKORU

75 / 100

1. Yerel Statik Analiz Bulgulari

✓ Yapılan yerel desen taramasında bilinen basit mimari zafiyet kalıplarına rastlanmadı.

2. Hermes AI Agent Derin Denetim ve Akıllı Yama

Yapay Zeka Skor Analizi: Gözden geçirildi.

75

Bu kontrat, WETH9 (Wrapped Ether) standardının temel bir uygulamasıdır. Genel olarak ERC-20 token konseptini Ether ile birleştirerek, Ether'in ERC-20 uyumlu tokenlar gibi işlem görmesini sağlar. Bu, DeFi protokollerinde ve diğer akıllı kontrat etkileşimlerinde oldukça yaygın bir modeldir. Kodun basitliği ve iyi bilinen bir standardı takip etmesi, bazı riskleri ortadan kaldırsa da, incelenmesi gereken kritik noktalar bulunmaktadır.

Kritik Risk Analizi ve Düzeltmeler

1. Reentrancy Zafiyeti (Yeniden Giriş Saldırıları)

Risk Açıklaması:

`withdraw` fonksiyonu, kullanıcı'nın bakiyesini düzürmeden önce `msg.sender.transfer(wad)` çağrısını yapmaktadır. Bu, bir saldırganın (kontrat olması durumunda) `transfer` fonksiyonu tarafından geri çağırıldığında (fallback fonksiyonu üzerinden), `balanceOf[msg.sender]` değeri güncellenmeden önce `withdraw` fonksiyonunu tekrar çağırabilmesine olanak tanır. Yani, saldırgan kontratın bakiyesinde kayıtlı olan Ether'i birden fazla kez çekebilir. Bu klasik bir Reentrancy saldırısı senaryosudur.

DUZELTILMIS GUVENLI SOLIDITY KODU:

```
``solidity
function withdraw(uint wad) public {
  require(balanceOf[msg.sender] >= wad, "Yetersiz bakiye"); // Girdi kontrolü
```

```
  balanceOf[msg.sender] -= wad; // Durum güncellemeleri önce yapılmalı
  (Checks-Effects-Interactions)
```

```
// Güvenli Ether transferi için send veya call kullanılması önerilir ve reentrancy guard ile korunmalıdır.
```

```
// Ancak bu durumda, transfer fonksiyonu reentrancy'nin ana kaynağı olduğu için,
```

```
// EIP-2384 veya ödeme desenleri (pull-over-push) kullanılabilir.
```

```
// Solidity 0.8.x ve sonrası için .transfer() kullanımı da gaz limiti nedeniyle reentrancy
```

riskini azaltır ancak tamamen ortadan kaldırmaz.

```
// Daha güvenli bir yaklaşım için 'call' ve reentrancy guard'ın kullanılması gereklidir.  
// Mevcut pratikte `.transfer()` kullanımı önerilmez.
```

```
// Örnek bir güvenli transfer yöntemi (Solidity 0.8.x varsayılarak):
```

```
(bool success, ) = msg.sender.call{value: wad}("");
```

```
require(success, "Ether transferi basarisiz oldu");
```

```
// YENİDEN GİRİŞ KORUMA DESENİ
```

```
// Bu kontrat için, kullanıcı Ether'i çektiğinde kendi kontratına yönlendirip tekrar withdraw çağırması temel sorundur.
```

```
// `Checks-Effects-Interactions` bu sorunu çözer. Durum güncellemelerini transferden önce yaptığımız için risk azalır.
```

```
// Ancak daha kapsamlı kontratlarda ReentrancyGuard gibi mekanizmalar kullanılabilir.
```

```
emit Withdrawal(msg.sender, wad);
```

```
}  
...
```

Açıklama:

`Checks-Effects-Interactions` deseni uygulanarak `balanceOf[msg.sender] -= wad` satır, Ether transferinden önceye alındı. Bu sayede, transfer çağrısı başka bir kontratı tetiklese bile, `balanceOf` değeri zaten güncellenmiş olacağından, saldırgan aynı fonları tekrar çekemeyecektir. Ayrıca, `transfer` yerine `call` kullanarak daha modern ve esnek bir transfer yöntemi sağlandı ve başarı olup olmadık kontrol edildi. `transfer` fonksiyonunun 2300 gas limiti reentrancy'yi zorlatırsa da, tamamen engellemez ve modern Solidity sürümlerinde deprecated kabul edilir.

2. Güvenlik iyileştirmeleri ve En İyi Uygulamalar (Eksiklikler)

Risk Açıklaması:

Kontratın belirtilen `pragma` versiyonu `solidity >=0.4.22 <0.6` aralığında. Bu eski versiyon aralığı, modern Solidity derleyicilerinin sağladığı güvenlik iyileştirmelerinden ve dahili denetimlerinden faydalanmamaktadır. Özellikle 'overflow' ve 'underflow' gibi aritmetik hatalar için varsayılan kontrollerin olmaması risk oluşturabilir. `uint` tipindeki değişkenlerde toplama veya çarpma işlemlerinde denetimsiz aşma/taşıma durumları fon kaybına yol açabilir.

DUZELTİLMİŞ GÜVENLİ SOLIDITY KODU:

```
```solidity
```

```
// pragma solidity ^0.8.0; // Modern ve güvenli bir Solidity versiyonu kullanılmalı.
```

```
// Kontratın içinde matematik işlemleri SafeMath kütüphanesi ile güvence altına alınmalıdır
```

```
// veya Solidity 0.8.0 ve üzeri kullanılarak varsayılan aşma/taşıma kontrollerinden faydalanılmalıdır.
```

```
// Bu kontratta `wad` ve `msg.value` gibi değişimler 'uint' olarak tanımlanmıştır.
```

```
contract WETH9 {
```

```
 string public name = "Wrapped Ether";
```

```
 string public symbol = "WETH";
```

```
 uint8 public decimals = 18;
```

```
 event Approval(address indexed src, address indexed guy, uint wad);
```

```
 event Transfer(address indexed src, address indexed dst, uint wad);
```

```

event Deposit(address indexed dst, uint wad);
event Withdrawal(address indexed src, uint wad);

mapping (address => uint) public balanceOf;
mapping (address => mapping (address => uint)) public allowance;

// ... (diğer fonksiyonlar önceki gibi) ...

function deposit() public payable {
// Solidity 0.8.0 ve üzeri kullanıldığında bu işlemler varsayılan olarak kontrol edilir.
// Eski versiyonlarda SafeMath gibi kütüphanelerle korunması gerekir.
// balanceOf[msg.sender] += msg.value;
// Eğer 0.6 veya daha düşük bir versiyon kullanılıyorsa SafeMath kullanılması şöyledir:
// balanceOf[msg.sender] = balanceOf[msg.sender].add(msg.value);
balanceOf[msg.sender] += msg.value; // Varsayarak 0.8.0+ kullanılıyor
emit Deposit(msg.sender, msg.value);
}

function withdraw(uint wad) public {
require(balanceOf[msg.sender] >= wad, "Yetersiz bakiye");

// balanceOf[msg.sender] -= wad;
// Eğer 0.6 veya

```